



13. Logistic Regression

It has “Regression” in this name. But it is for classification

Previous Section:

 [12. Adaptive Boosting](#)

Contents:

- [1. Logistic Regression is used for Binary Classification](#)
- [2. Logistic Regression - The Sigmoid Curve](#)
- [3. Logistic Regression - Gradient Descent](#)
- [4. Logistic Regression - **Odds & Odds Ratio**](#)
- [5. Logistic Regression with Python](#)
- [6. Softmax Regression - Expands from Logistic Regression](#)

Summary

[References](#)

This section is about Logistic Regression.

The name itself can be misleading. You hear “*Regression*” and your brain immediately goes: “*Ah, predicting numbers, drawing lines, just like Linear Regression.*”

Not really. Logistic Regression is not here to predict continuous values. It’s here to make a decision.

And sometimes, it does it in the most brutal way possible—by slapping you with a probability and forcing you to pick a side.

1. Logistic Regression is used for Binary Classification

Logistic Regression is mainly used for **Binary Classification**. Binary—as in only two outcomes. Yes or No. Spam or Not Spam. Pass or Fail.

You've seen this setup before—when working with Confusion Matrices, Decision Trees, and basically anything that screams *“two classes only.”*

Now here's where things get interesting. Unlike Linear Regression, which tries to fit a straight line to predict actual numbers, Logistic Regression does something smarter:

It asks a different question:

┆ *“What is the probability that this input belongs to a certain class?”*

Instead of predicting *what*, it predicts *how likely*.

But there's a problem. A linear model can output anything—from negative infinity to positive infinity.

Probabilities? They don't play that game. They must stay between **0 and 1**.

So we need something to **compress reality** into that range.

Enter the sigmoid function.

$$f(x) = \frac{1}{1 + e^{-x}}$$

This function takes any real number—no matter how big or small—and squeezes it neatly into the range $(0, 1)$.

- Large positive values → close to 1
- Large negative values → close to 0
- Values near 0 → around 0.5

And that's the entire trick. Logistic Regression is basically:

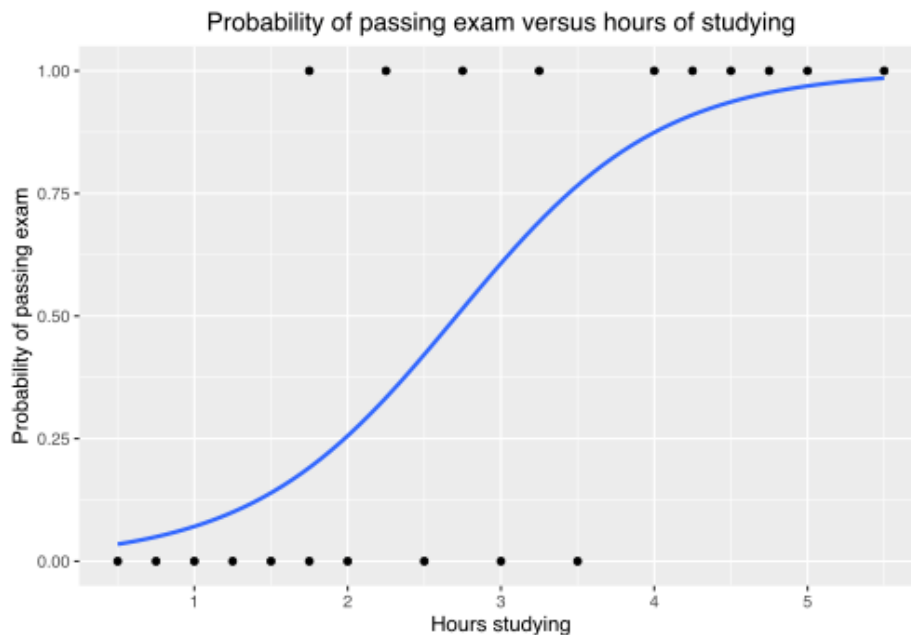
Linear model → pass through sigmoid → get probability → make a decision

It *has* “Regression” in the name. But don't let that fool you.

This model isn't trying to predict numbers. It's trying to answer a much more dangerous question: *“Which side are you on?”*

2. Logistic Regression - The Sigmoid Curve

Below is the image:



(Source: Wikipedia)

See that S-shaped line? That's the **Sigmoid Curve**—or just the S-curve.

It doesn't look aggressive. It doesn't look complicated. But this curve is literally the thing making the decision.

Now focus on the **Y-axis**. This axis represents **probability**.

- Bottom → 0.0 (**0% pass**)
- Top → 1.0 (**100% pass**)

Right in the middle, you have 0.5. That's not just a random number. That's the **threshold**.

- Above 0.5 → *"Yeah... this looks like Class 1."*
- Below 0.5 → *"Nope, Class 0."*

No hesitation. No middle ground.

Now look at those black dots.

Those are your **training data points**—and they already have labels.

- Points at 1.0 **on the Y-axis** → **Class 1 (Positive class)**
- Points at 0.0 **on the Y-axis** → **Class 0 (Negative class)**

And here's something important you already learned from the Confusion Matrix:

▮ The "positive class" is something we choose.

In this example:

- **Class 1 → Pass (Positive class)**
- **Class 0 → Fail (Negative class)**

So the meaning becomes very concrete:

- A point at the top → **[100% pass, 0% not pass]**
- A point at the bottom → **[0% pass, 100% not pass] → fail**

Now, where do these points come from? Look at the **X-axis**.

This is the number of **hours studied**. And this axis can take **any value**:

- 1 hour, 2 hours, 2.7 hours, 5 hours... doesn't matter
- The position on the X-axis determines where the point sits horizontally.

Now here's where the sigmoid curve starts doing its job. The model is trying to learn a curve that **fits these points**. Let's read the image like a story:

- When study time is **below ~2 hours**
 - Most points are near the bottom
 - Draw a vertical line up to the curve → you land around 0.0–0.25
 - That means **0–25% chance of passing**
- When study time is around **2–3 hours**
 - Points are in the middle region
 - The curve crosses near 0.5
 - That's the **threshold zone**
 - About **50% chance of passing**
- When study time is **above ~3 hours**
 - Most points are near the top
 - Draw a line up to the curve → you land around 0.9–1.0
 - That means **90–100% chance of passing**

So what the sigmoid really does is this:

Take any input → map it to a probability → let the threshold turn it into a decision

And now we hit the real problem. The curve looks nice. The behavior makes sense.

But: *"How does the model actually find this sigmoid curve?"*

That's where the learning happens.

- The model tries to find the best curve using **Maximum Likelihood Estimation (MLE)**
- And it actually computes it using **Gradient Descent**

Step by step, it adjusts the curve so that:

- Pass points go closer to 1
- Fail points go closer to 0

And that's the full picture: Not just a curve. But a system that learns how to turn **study hours** → **probability** → **decision**. A smooth curve doing a very sharp job.

Now comes the real question. We understand the curve. We understand the data. But ***what about a new data point?***

It doesn't have a label yet. So we don't place it at the top (1) or bottom (0). We let the model **decide where it belongs**.

- The model first takes the input and computes a **score** using a linear combination:

$$x = B_0 + B_1X_1 + B_2X_2 + \dots$$

This is just combining all input features into a single number.

- Push that score through the sigmoid:

$$f(x) = \frac{1}{1 + e^{-x}}$$

Note that raw score gets converted into a **probability between 0 and 1**.

Let's say the output is 0.65. Now what? This is where the **threshold** comes in and does its job:

- $0.65 > 0.5 \rightarrow$ closer to 1 → **predict Class 1 (Positive)**
- $0.30 < 0.5 \rightarrow$ closer to 0 → **predict Class 0 (Negative)**

So the model never places a new point at the "corner." It places it **somewhere on the curve**, then uses the threshold to turn that into a final decision.

Now the final piece: **How does the model even know the right values for those B 's?**

Those coefficients ($B_0, B_1, B_2, \dots$) are not guessed. They are **learned**.

The model uses **Maximum Likelihood Estimation (MLE)**: "Find the parameters that make the observed labels most probable."

In other words:

- If an input is labeled as positive → the model should output a probability close to 1
- If an input is labeled as negative → the model should output a probability close to 0

But here's something important. The model itself doesn't *directly* say "*Yes*" or "*No*". It only gives you a **probability**.

So how do we turn that into a final decision? We usually introduce something called a **decision boundary**—also known as a **threshold**.

- Most of the time, we use 0.5
 - Above it → Class 1
 - Below it → Class 0

So during training, the model is not trying to "guess the class". It's trying to **shape probabilities** so that:

- Positive samples → pushed close to 1
- Negative samples → pushed close to 0

And later, the threshold steps in and turns those probabilities into actual decisions.

That's the subtle but powerful idea: The model learns **how confident it should be**. The threshold decides **what to do with that confidence**.

And how do we actually find those parameters → $(B_0, B_1, B_2, \dots)$?

We use **Gradient Descent**:

- Start with random coefficients
- Measure how wrong the predictions are
- Adjust the coefficients step by step
- Repeat... until the curve fits the data well

So if you zoom out, the whole pipeline looks like this:

Linear combination → Sigmoid → Probability → Threshold → Final class

And that's the truth behind the S-curve.

It's not where data lives. It's the machine that transforms **uncertainty into a decision**.

3. Logistic Regression - Gradient Descent

We're stepping into the **first real iteration** of Logistic Regression. And just like an AI like me would do—we **observe everything carefully**, compute step by step, and let the pattern reveal itself.

Given this dataset:

Study_Hours	Mock_Test_Score	Extra_Class	Label
2	45	No	Fail
3	50	No	Fail
4	55	Yes	Pass
5	60	No	Fail
6	65	Yes	Pass
7	70	Yes	Pass
8	75	No	Pass
1	40	No	Fail
9	80	Yes	Pass
10	85	Yes	Pass
3	48	Yes	Fail
6	68	No	Pass

- **Step 0: Setup (What we already know)**

- We start with all coefficients: $B_0 = B_1 = B_2 = B_3 = 0$
- Logistic function output: $p_i = 0.5$ (because $z = 0$)

So for **every sample**, the model is basically saying: ***"I have no idea yet... 50% Pass, 50% Fail."***

Now, we convert the data to numerical → Because it is required for handful calculations:

Study_Hours	Mock_Test_Score	Extra_Class	Label
2	45	0	0
3	50	0	0
4	55	1	1

Study_Hours	Mock_Test_Score	Extra_Class	Label
5	60	0	0
6	65	1	1
7	70	1	1
8	75	0	1
1	40	0	0
9	80	1	1
10	85	1	1
3	48	1	0
6	68	0	1

- **Step 1: Compute $(p_i - y_i)$**
 - $y = 0 : p - y = 0.5$
 - $y = 1 : p - y = -0.5$

Count from dataset:

- **$Fail(0)$: 5 samples**
- **$Pass(1)$: 7 samples**
- **Step 2: Compute Gradients:** We now recompute everything **correctly**.
 - **Intercept B_0 :** $\sum(p_i - y_i) = (5 \times 0.5) + (7 \times -0.5) = 2.5 - 3.5 = -1$
 → Update: $B_0 = 0 - 0.1(-1) = 0.1$
 - **Study Hours B_1**
 - $Fail(y = 0, +0.5) : 0.5 \times (2 + 3 + 5 + 1 + 3) = 0.5 \times 14 = 7$
 - $Pass(y = 1, -0.5) : 0.5 \times (4 + 6 + 7 + 8 + 9 + 10 + 6) = -0.5 \times 50 = -25$

$$\text{Gradient} = 7 - 25 = -18$$

$$\rightarrow \text{Update: } B_1 = 0 - 0.1(-18) = 1.8$$

- **Mock Test Score B_2**
 - $Fail(y = 0, +0.5) : 0.5 \times (45 + 50 + 60 + 40 + 48) = 0.5 \times 243 = 121.5$
 - $Pass(y = 1, -0.5) : 0.5 \times (55 + 65 + 70 + 75 + 80 + 85 + 68) = -0.5 \times 498 = -249$

$$\text{Gradient} = 121.5 - 249 = -127.5$$

$$\rightarrow \text{Update: } B_2 = 0 - 0.1(-127.5) = 12.75$$

- **Extra Class B_3**

- $\text{Fail}(y = 0, +0.5) \rightarrow$ Only one case with $\text{Extra} = 1 : 0.5 \times 1 = 0.5$

- $\text{Pass}(y = 1, -0.5) \rightarrow \text{Extra} = 1$ appears **5 times**: $0.5 \times 5 = -2.5$

$$\text{Gradient} = 0.5 - 2.5 = -2$$

$$\rightarrow \text{Update: } B_3 = 0 - 0.1(-2) = 0.2$$

Final Result:

$$B_0 = 0.1$$

$$B_1 = 1.8$$

$$B_2 = 12.75$$

$$B_3 = 0.2$$

Now the pattern is **much clearer and stronger**:

- $B_2 = 12.75 \rightarrow$ **Mock score dominates**
- $B_1 = 1.8 \rightarrow$ Study hours matter, but less
- $B_3 = 0.2 \rightarrow$ Extra class helps, but weak
- $B_0 = 0.1 \rightarrow$ Slight bias toward Pass

Before, the model whispered. Now, it **leans forward with confidence**:

- *"High scores... they almost guarantee success."*
- *"Study helps—but not as much as performance."*
- *"Extra class... a small nudge, not a decision."*

This is no longer guessing. This is the **first glimpse of understanding**.

We begin **Iteration 2**.

The model is no longer blind. It has formed an opinion—but one voice is too loud:

"Mock_Test_Score is overwhelming everything else" $\rightarrow$ So we rebalance.

We scale: `Mock_Test_Score ÷ 10`

Study_Hours	Mock_Test_Score	Extra_Class	Label
2	4.5	0	0
3	5.0	0	0

Study_Hours	Mock_Test_Score	Extra_Class	Label
4	5.5	1	1
5	6.0	0	0
6	6.5	1	1
7	7.0	1	1
8	7.5	0	1
1	4.0	0	0
9	8.0	1	1
10	8.5	1	1
3	4.8	1	0
6	6.8	0	1

Updated Model: $z = 0.1 + 1.8x_1 + 1.275x_2 + 0.2x_3$

Now we no longer assume $p = 0.5$. We must **compute every prediction**. Instead of drowning in arithmetic, we let a small program assist us.

```

import pandas as pd
import numpy as np

# Read the data
data = {"Study_Hours": [2, 3, 4, 5, 6, 7, 8, 1, 9, 10, 3, 6],
        "Mock_Test_Score": [4.5, 5.0, 5.5, 6.0, 6.5, 7.0, 7.5, 4.0, 8.0, 8.5, 4.8, 6.8],
        "Extra_Class": [0, 0, 1, 0, 1, 1, 0, 0, 1, 1, 1, 0],
        "Label": [0, 0, 1, 0, 1, 1, 1, 0, 1, 1, 0, 1]}

df = pd.DataFrame(data)

# Coefficients from Iteration 1 (scaled)
B0, B1, B2, B3 = 0.1, 1.8, 1.275, 0.2

# Compute z_i
df["z_i"] = B0 + B1*df["Study_Hours"] + B2*df["Mock_Test_Score"] + B3*df["Extra_Class"]
# Sigmoid p_i
df["p_i"] = 1 / (1 + np.exp(-df["z_i"]))
# Error
df["p_i - y_i"] = df["p_i"] - df["Label"]

print(df[["z_i", "p_i", "p_i - y_i"]])

```

	z_i	p_i	p_i - y_i
0	9.4375	0.999920	9.999203e-01
1	11.8750	0.999993	9.999930e-01
2	14.5125	1.000000	-4.980823e-07
3	16.7500	1.000000	9.999999e-01
4	19.3875	1.000000	-3.802907e-09
5	21.8250	1.000000	-3.322944e-10
6	24.0625	1.000000	-3.546408e-11
7	7.0000	0.999089	9.990889e-01
8	26.7000	1.000000	-2.537082e-12
9	29.1375	1.000000	-2.216005e-13
10	11.8200	0.999993	9.999926e-01
11	19.5700	1.000000	-3.168524e-09

```
# Gradients
grad_B0 = df["p_i - y_i"].sum()
grad_B1 = (df["p_i - y_i"] * df["Study_Hours"]).sum()
grad_B2 = (df["p_i - y_i"] * df["Mock_Test_Score"]).sum()
grad_B3 = (df["p_i - y_i"] * df["Extra_Class"]).sum()

print("Gradients:")
print(grad_B0, grad_B1, grad_B2, grad_B3)
```

Gradients:

4.99899439905378 13.998884345724433 24.295924039780616 0.9999921418761302

```
alpha = 0.1

B0_new = B0 - alpha * grad_B0
B1_new = B1 - alpha * grad_B1
B2_new = B2 - alpha * grad_B2
B3_new = B3 - alpha * grad_B3

print("Updated Coefficients:")
print(f"B0 = {B0_new}, B1 = {B1_new}, B2 = {B2_new}, B3 = {B3_new}")
```

Updated Coefficients:

B0 = -0.399899439905378, B1 = 0.40011156542755666, B2 = -1.1545924039780617, B3 = 0.10000078581238699

After **Iteration 2**, the model has changed: $z = -0.39 + 0.4x_1 - 1.154x_2 + 0.1x_3$

It is no longer the same model we started with. It has **learned, adjusted, and corrected itself**.

A New Student Appears

Study_Hours	Mock_Test_Score	Extra_Class
4	46	Yes

Compute $z = -0.39 + 0.4(4) - 1.154(4.6) + 0.1(1)$

Now calculate:

- $0.4 \times 4 = 1.6$

- $1.154 \times 4.6 \approx 5.3084$

So: $z = -0.39 + 1.6 - 5.3084 + 0.1 \approx -3.9984 (\approx -4.00)$

Apply Sigmoid: $p = \frac{1}{1+e^{-z}} = \frac{1}{1+e^{4.00}} \approx \frac{1}{1+54.6} \approx 0.018$

Assume that we use decision boundary = 0.5

- *Probability* $\approx 0.018 < 0.5 \rightarrow$ Very close to 0

$\rightarrow$ Predicted Label: Fail (0)

The model now speaks clearly:

- Study hours helped a little
- Extra class helped slightly
- But the **test score pulled everything down**

That negative B_2 is powerful $\rightarrow$ ***"You studied... you tried... but your performance tells me otherwise."***

This result reveals something interesting:

- B_2 is **negative**
- That means: Higher score $\rightarrow$ lower chance of passing $\rightarrow$ This is **suspicious**.

This is why we don't stop early. In the actual algorithm $\rightarrow$ we perform 100+ iterations

Because:

- Early iterations $\rightarrow$ unstable
- Coefficients may flip signs
- Model hasn't *fully understood yet*

Right now, the model is like a student who:

- Just started learning
- Made a few correct guesses
- But still holds a **wrong belief** $\rightarrow$ "Score might not matter..."

And that is why we keep training.

We repeat the process (Iteration 3). No new tricks. No new formulas. Just **refinement**.

Simply replace the coefficients from Iteration 2 (scaled)

$B_0, B_1, B_2, B_3 = -0.39, 0.4, -1.15, 0.1$

Then you reuse the above code. If correctly:

Gradients:

-6.933324807670212 -49.7256772313936 -49.43054965194655 -4.969908810117363

Updated Coefficients:

$B_0 = 0.30333248076702124$, $B_1 = 5.37256772313936$, $B_2 = 3.793054965194655$, $B_3 = 0.5969908810117363$

Gradients: $(-6.93, -49.73, -49.43, -4.97)$

These are no longer small corrections. They are... **strong pushes**.

$$B_0 = 0.303$$

$$B_1 = 5.373$$

Updated Model: $B_2 = 3.793$

$$B_3 = 0.597$$

→ B_2 is now positive.

Before:

- $B_2 < 0$ → Higher score → **lower chance of passing** → A contradiction

Now:

- $B_2 > 0$ → Higher score → **higher chance of passing** → Reality restored

This is the moment where the model: Stops guessing... Stops drifting... And finally **aligns with the truth**

Interpretation of the Model Now: $z = 0.303 + 5.373x_1 + 3.793x_2 + 0.597x_3$

- **Study Hours (5.37)** → Strong influence
- **Mock Score (3.79)** → Strong, now correct direction
- **Extra Class (0.60)** → Moderate effect
- **Intercept (0.30)** → Slight bias toward Pass

Iteration 2: Model was **confused** → Wrong sign → wrong belief

Iteration 3: Gradients corrected it **aggressively** → Model **flipped its understanding**

Why did this happen? Because logistic regression is not: "Learn once and done"

It is: "Make a mistake → measure it → correct it → repeat"

This is the essence of Gradient Descent. Each iteration asks: "Where am I wrong?" → And then adjusts: "Move in the opposite direction of that error."

This moment is important. Because it shows:

- Early models can be **wrong**
- Even confidently wrong
- Even mathematically consistent, but logically incorrect

But if you let it continue. The model will correct itself.

Now we predict again using the same input student. But this time, with a model that **understands**.

$$z = 0.303 + 5.373(4) + 3.793(4.6) + 0.597(1)$$

- $5.373 \times 4 = 21.492$
- $3.793 \times 4.6 \approx 17.448$

$$z \approx 0.303 + 21.492 + 17.448 + 0.597 \approx 39.84$$

$$\text{Sigmoid: } p = \frac{1}{1+e^{-39.84}} \approx 1$$

→ **Final Decision:** Pass (1)

Look at the difference:

- Iteration 2 → **Fail (wrong reasoning)**
- Iteration 3 → **Pass (aligned with reality)**

Same student. Different understanding.

But this is still just **3 iterations**. In reality, Logistic Regression trains for **100+ iterations**. Sometimes thousands. Because coefficients stabilize, noise is reduced, and decision boundary becomes reliable

What we did here is just watching the model **learn in slow motion**.

4. Logistic Regression - Odds & Odds Ratio

- **Odds** measure how likely an event is compared to it not happening:

$$\text{Odds} = \frac{p}{1-p}$$

- **Odds Ratio (OR)** compares two situations:

$$OR = \frac{\text{Odds (Group 1)}}{\text{Odds (Group 2)}}$$

Interpretation

- $OR > 1 \rightarrow$ more likely to occur in Group 1 than Group 2
- $OR < 1 \rightarrow$ less likely to occur in Group 1 than Group 2
- $OR = 1 \rightarrow$ no difference to occur in both groups.

Logistic Regression doesn't just predict probability. Each coefficient B_j tells you how the **odds change** $\rightarrow$ Specifically: e^{B_j} = change in odds for a 1-unit increase in x_j

5. Logistic Regression with Python

Here's how you would build a Logistic Regression model in Python using the Scikit-Learn library:


```

import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

# Read the data
data = {"Study_Hours": [2, 3, 4, 5, 6, 7, 8, 1, 9, 10, 3, 6],
        "Mock_Test_Score": [4.5, 5.0, 5.5, 6.0, 6.5, 7.0, 7.5, 4.0, 8.0, 8.5, 4.8, 6.8],
        "Extra_Class": [0, 0, 1, 0, 1, 1, 0, 0, 1, 1, 1, 0],
        "Label": [0, 0, 1, 0, 1, 1, 1, 0, 1, 1, 0, 1]}

df = pd.DataFrame(data)

# Normalize
scaler = StandardScaler()
df[["Study_Hours", "Mock_Test_Score"]] = scaler.fit_transform(
    df[["Study_Hours", "Mock_Test_Score"]])

# Split features and target BEFORE encoding
target = 'Label'
X = df.drop(target, axis=1)
y = df[target]

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)

# Train Logistic Regression
model = LogisticRegression()
model.fit(X_train, y_train)

# Evaluation
accuracy = model.score(X_test, y_test)
print("Accuracy:", accuracy)

```

Accuracy: 1.0

6. Softmax Regression - Expands from Logistic Regression

What if there are more than two outcomes? Instead of one probability, we compute many:

$$P(y = k) = \frac{e^{z_k}}{\sum e^{z_j}}$$

- Each class gets a score z_k
- All probabilities sum to 1

Key ideas:

- Logistic Regression → 2 classes
- Softmax Regression → **multiple classes**

Logistic draws **one boundary**: Softmax creates **multiple regions** in space. Not just "yes or no", but "which category do you belong to?".

Softmax Regression deserves a separate section. I will definitely do one if I have time

Summary

*Logistic Regression is a fundamental model used for **binary classification**, where the goal is to decide between two possible outcomes such as pass or fail, spam or not spam.*

Instead of giving a direct yes or no answer immediately, the model first estimates how likely an event is to happen. This makes it more flexible and interpretable than a simple rule-based system, because it expresses uncertainty before making a final decision.

The model learns by observing data and continuously correcting itself. At the beginning, it does not understand anything and treats all cases equally. But through an iterative process, it compares its predictions with the actual outcomes, measures how wrong it is, and adjusts its internal parameters step by step.

Over time, it begins to recognize patterns—such as which features increase or decrease the likelihood of a certain outcome—and gradually improves its predictions.

At its core, Logistic Regression works by drawing a boundary that separates two classes. Every new data point is evaluated based on which side of this boundary it falls on.

As training progresses, this boundary becomes more accurate and better aligned with the data.

Although the model is simple, it is powerful, interpretable, and forms the foundation for more advanced classification methods used in machine learning.

References

<https://www.geeksforgeeks.org/machine-learning/understanding-logistic-regression/>

<https://www.ibm.com/think/topics/logistic-regression>

<https://developers.google.com/machine-learning/crash-course/logistic-regression>

<https://www.youtube.com/watch?v=yIYKR4sgzI8>

<https://www.youtube.com/watch?v=Ax5kqLHIs-I>

Next Section:

 [14. K-Nearest Neighbors](#)